Aplicativo de acessibilidade com uso de eye-tracking

Miguel C. de Medeiros Thiago C. R. Rabelo

Orientador: Prof. Dr. Edson Mintsu – ENE/UnB

Departamento de Ciência da Computação
Universidade de Brasília
Curso de Engenharia de Computação



Problemas Enfrentados

Esclerose Lateral Amiotrófica (ELA)

- **O que é**
  - Doença neurodegenerativa progressiva dos neurônios motores [2]
  - Fraqueza e paralisia gradual dos músculos voluntários

- **Impacto na comunicação**
  - A fala se deteriora até ser perdida
  - Mãos e braços também perdem movimento: escrever e digitar deixam de ser opção
  - A cognição costuma ser preservada: a pessoa continua tendo o que dizer

- **Movimento ocular preservado**
  - Os músculos dos olhos costumam ser poupados até fases avançadas
  - O olhar se torna o principal canal de comunicação

Soluções Existentes

Smartbox [12]

- Câmera infravermelha Lumin-i acoplada ao tablet
- Tolera movimentos da cabeça e luz do sol
- Grid 3: símbolos, teclado com predição e voz





Tobii Dynavox [15]

- Sensor infravermelho embutido (reflexo na córnea e pupila)
- Tela traseira mostra o texto ao interlocutor
- TD Snap e Communicator 5 para comunicação

Soluções Existentes

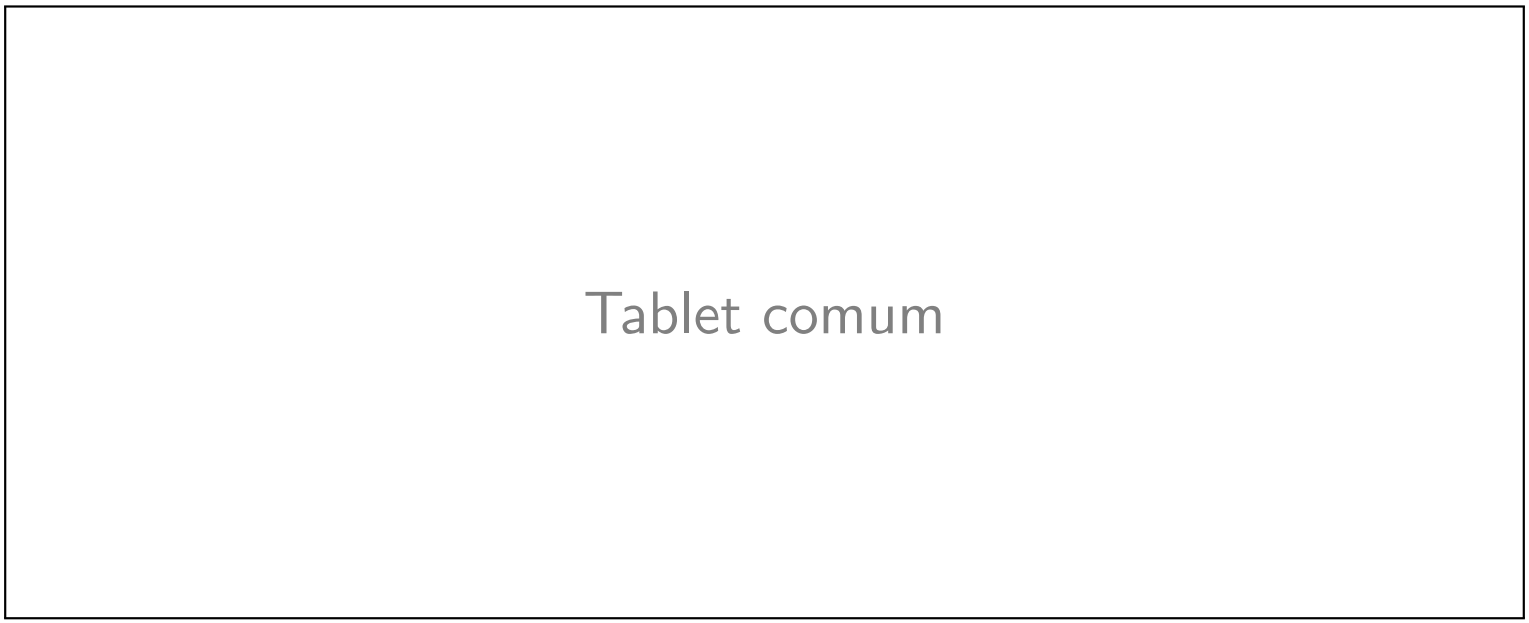
O que oferecem

- Rastreamento preciso, mesmo sob luz do sol e com movimentos da cabeça
- Comunicação completa: símbolos, teclado com predição e voz
- Controle do ambiente
- Aparelho resistente, para cadeira de rodas ou cama

Mas a que custo?

- Exigem câmera infravermelha e tablet dedicados
- Grid Pad 13 com Lumin-i: US$ 16.485 [13]
- TD I-13: US$ 15.565 [1]
- Fora do alcance da maioria das famílias, ainda sem impostos de importação

Nosso Objetivo

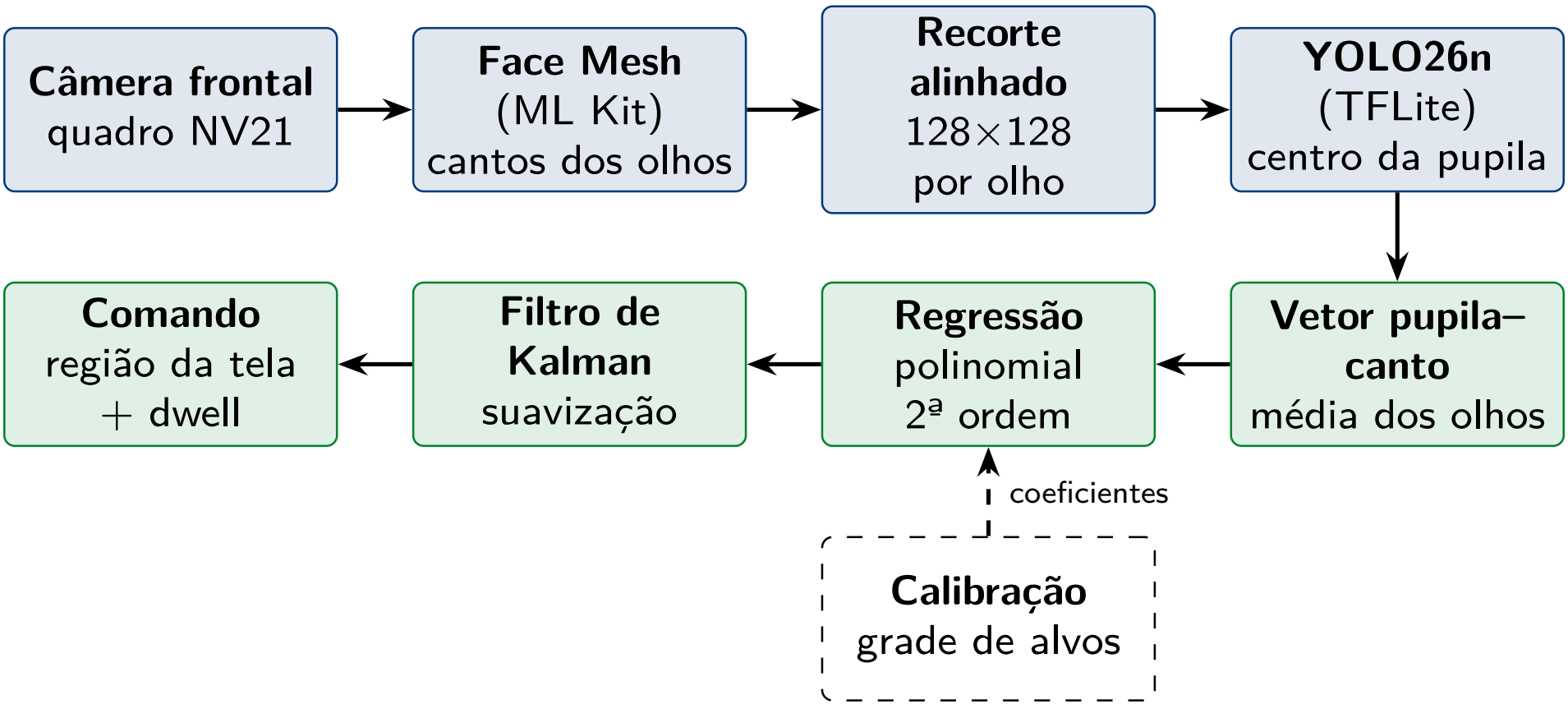


Objetivo

Sistema de rastreamento ocular com sistema de teclado com predição e voz que possa ser utilizado em aparelhos tablet comuns.

Eye-tracking

Abordagem

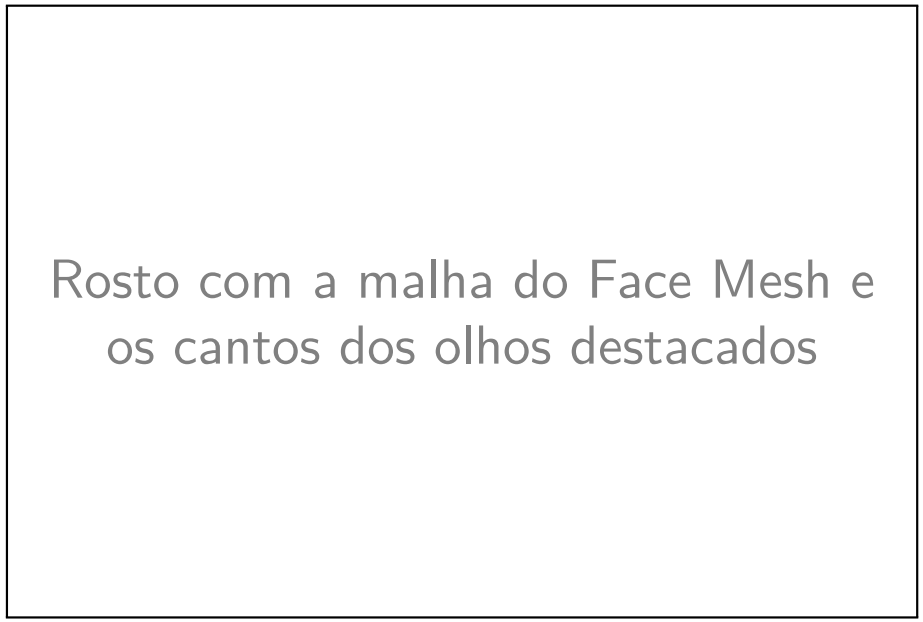


Eye-tracking

Rosto e Olhos: ML Kit Face Mesh

ML Kit Face Mesh [4]

- SDK de aprendizado de máquina do Google para celulares, com inferência no próprio aparelho
- Face Mesh: 468 pontos 3D do rosto em tempo real



No sistema

- Localiza o rosto e os cantos de cada olho
- Os cantos definem o recorte do olho e a referência do vetor pupila–canto
- Não marca a pupila: daí o segundo modelo

Eye-tracking

Pupila: YOLO

YOLO26n [11, 16]

Detector de objetos que prevê as caixas numa única passada da rede.
Versão nano da Ultralytics, voltada a dispositivos sem GPU.

MPIIGaze [17]

- 213.659 imagens de 15 participantes, capturadas pela webcam no uso diário do notebook
- 10.654 imagens com o centro da pupila anotado à mão

Treinamento

- A partir dos pesos pré-treinados do YOLO26n
- 21.308 recortes de olho de 128×128 px, com a mesma geometria do app
- Separação por participante: 11 no treino, 2 na validação, 2 no teste

Teste em participantes nunca vistos

Pupila encontrada em 100% dos 2.332 recortes, com erro mediano de 4,5% da largura do olho (2,6 px num quadro de 1280×720).

Eye-tracking

Regressão

Entrada: vetor pupila–canto

$$\mathbf{v} = \frac{\mathbf{p} - \frac{1}{2}(\mathbf{c}_A + \mathbf{c}_B)}{\|\mathbf{c}_B - \mathbf{c}_A\|}$$

$\mathbf{p}$: centro da pupila; $\mathbf{c}_A, \mathbf{c}_B$: cantos do olho. Média dos dois olhos $\Rightarrow (v_x, v_y)$. Invariante a escala e translação: compensa parte do movimento da cabeça.

Modelo: polinômio de 2ª ordem

$$x_{\text{tela}} = a_0 + a_1 v_x + a_2 v_y + a_3 v_x^2 + a_4 v_y^2 + a_5 v_x v_y$$

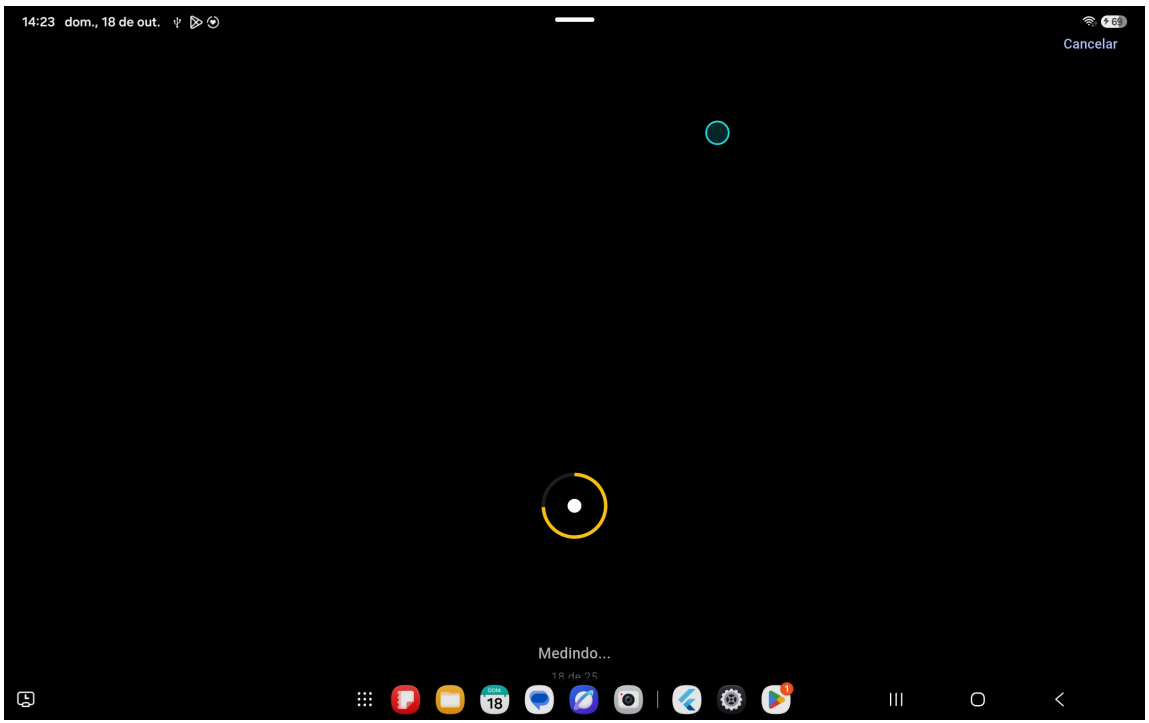
Mesma forma para y_{tela}: 6 coeficientes por eixo. Saída em fração da tela, de 0 a 1.

Eye-tracking

Calibragem

Ajuste na calibração

- Grade de 5×5 alvos: mediana de $\mathbf{v}$ em cada alvo
- Mínimos quadrados com regularização ridge, resolvido por eliminação de Gauss
- Descarta até 2 alvos mal fixados, enquanto o erro mediano diminuir



Eye-tracking

Filtro de Kalman

Motivação

- O ponto da regressão treme a cada quadro: ruído na pupila e nos cantos do olho
- O dwell exige 1,5 s na mesma região: o tremor reinicia a contagem

No sistema

- Um filtro por eixo, x e y independentes
- Variância da medida $R = 0{,}075$ e da aceleração $\sigma_a^2 = 1$
- Δt real entre quadros, limitado a 250 ms
- Reiniciado a cada nova calibração

Modelo de velocidade constante [8]

Estado $\mathbf{x} = (\text{posição}, \text{velocidade})$; a aceleração entra como ruído.

Predição:

$$\hat{\mathbf{x}}^- = F\hat{\mathbf{x}}, \qquad F = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix}$$

$$P^- = FPF^\top + Q$$

Correção:

$$K = \frac{P^- H^\top}{HP^- H^\top + R}, \qquad H = \begin{pmatrix} 1 & 0 \end{pmatrix}$$

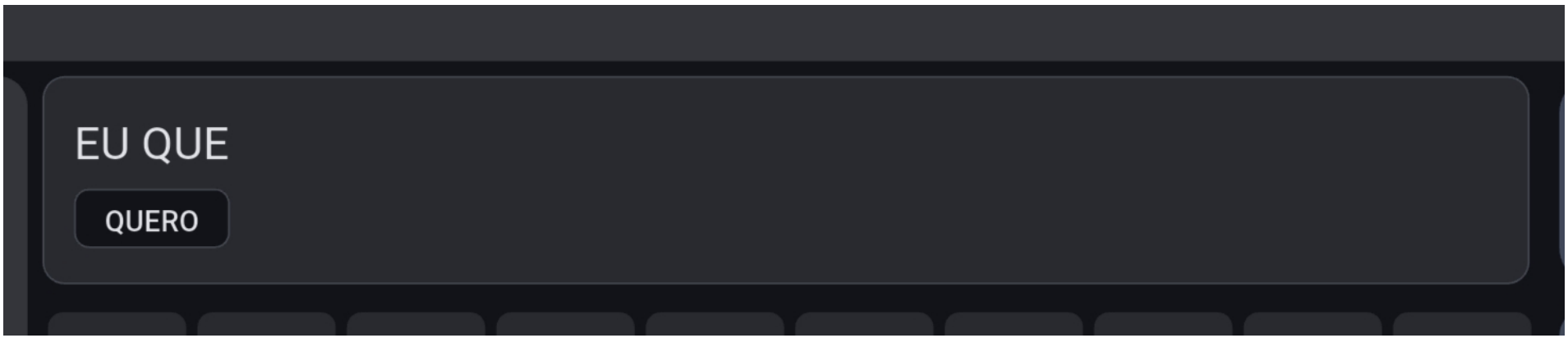
$$\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + K\left(z - H\hat{\mathbf{x}}^-\right)$$

$$P = (I - KH)\, P^-$$

z: ponto medido pela regressão. O ganho K pondera a previsão e a medida conforme a incerteza de cada uma.

Autocomplete

Abordagem



Do texto às sugestões

1. Normaliza: minúsculas, sem acento
2. Separa as palavras anteriores da palavra em digitação
3. Pontua o vocabulário com trigramas
4. Mostra as 3 melhores, com acento

Seleção pelo olhar

Da primeira linha do teclado, olhar para cima entra na barra de sugestões

Autocomplete

Modelo de Linguagem: n-gramas

Corpus

- 565 frases típicas de comunicação alternativa, cerca de 765 palavras
- Marcador de início de frase: “eu” no começo é um contexto diferente de “eu” no meio

Trigramas

- Conta cada palavra depois das 2 anteriores
- Contexto nunca visto: recua para 1 palavra e depois para nenhuma

Stupid Backoff [3]

$$S(w \mid h) = \begin{cases} \dfrac{c(h\,w)}{c(h)} & \text{se } c(h\,w) > 0 \\ 0{,}4 \cdot S(w \mid h') & \text{caso contrário} \end{cases}$$

h: palavras anteriores; h': h sem a mais antiga; c: contagem no corpus. Não é uma probabilidade, mas basta para ordenar as sugestões.

Autocomplete

Completar ou Sugerir a Próxima Palavra

- **Termina em espaço:** sugere a próxima palavra
  `EU QUERO␣` $\rightarrow$ BEBER · COMER · IR
- **Palavra pela metade:** completa
  `EU QUERO B` $\rightarrow$ BEBER · BOCA · BANHEIRO
- **Ambíguo:** `E` é a palavra "e" ou o começo de "estou"?

Barra com `E` $\rightarrow$ ESTOU · EU · A

As duas leituras competem

Completar: $S(c \mid h)$ Próxima palavra: $S(\text{e} \mid h) \cdot S(n \mid h\ \text{e})$

`E` $\rightarrow$ ESTOU · EU (completam) · A (vem depois)

Autocomplete

Tolerância a Erros de Digitação

Problema

Uma seta a mais ou a menos troca a letra pela vizinha: `BWB` em vez de `BEB`, e nenhuma palavra começa assim

Distância de edição [9]

- Até o *prefixo* mais próximo
- Vizinha, letra a mais ou a menos: 1; tecla distante: 2
- Até 1 erro, a partir de 3 letras
- Pontuação $\times 0{,}01$: as exatas vêm primeiro

Barra com `EU QUERO BWB` → BEBER

Autocomplete

Avaliação Offline

Método

- Digitador simulado escreve as frases do corpus pelo caminho mínimo de setas e aceita a sugestão quando ela custa menos comandos
- Validação cruzada em 10 partes: o modelo nunca viu a frase testada
- Mesmas métricas das sessões com pessoas [14]

Sem erros de digitação	
Comandos poupados	50,7%
Palavras entre as 3	78,9%
Fora do vocabulário	16,5%

10% de letras erradas*	
Sem tolerância	30,6%
Com tolerância	36,6%

* comandos poupados; erro notado só no fim da palavra

Geração de Áudio

Tecnologias

Kokoro [6]

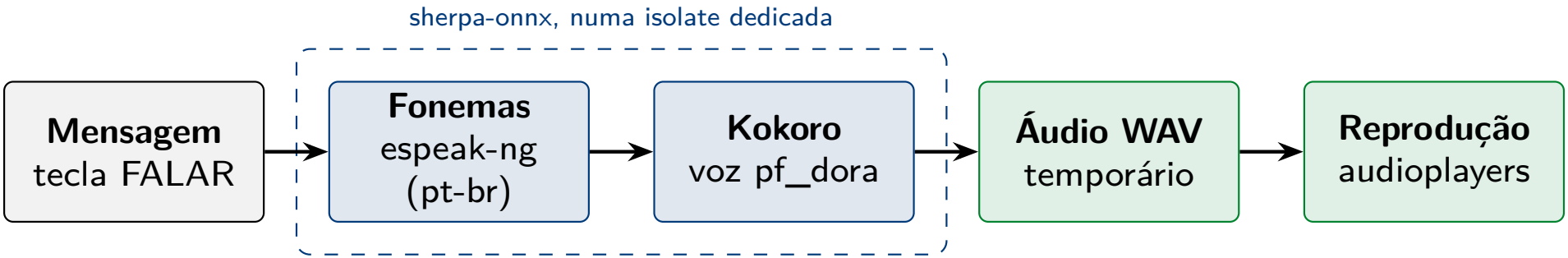
- Modelo de síntese de fala de pesos abertos, com 82 milhões de parâmetros
- Arquitetura derivada do StyleTTS 2 [10]: converte fonemas em áudio numa única passada da rede
- Versão 1.0: 8 idiomas e 54 vozes, incluindo português brasileiro
- Licença Apache 2.0

sherpa-onnx [7]

- Biblioteca do projeto Next-gen Kaldi para rodar modelos de fala sem internet
- Executa os modelos com o ONNX Runtime
- Suporta Android e oferece pacote para Flutter
- Licença Apache 2.0

Geração de Áudio

No sistema



Decisões

- Totalmente offline: a frase não sai do aparelho
- Isolate dedicada: a síntese não trava a interface nem o rastreador de olhar
- Uma fala por vez: dwells repetidos não sobrepõem áudios
- Modelo de ~355 MB fora do Git, baixado por script e verificado por SHA-256

Stack

Flutter [5]

- Plugins prontos para tudo que o sistema usa: câmera, ML Kit, TFLite e sherpa-onnx
- Quadros da câmera em tempo real, sem escrever código nativo
- Isolates: a síntese de voz roda sem travar a interface
- Hot reload e testes integrados aceleram os ajustes de interface e calibração

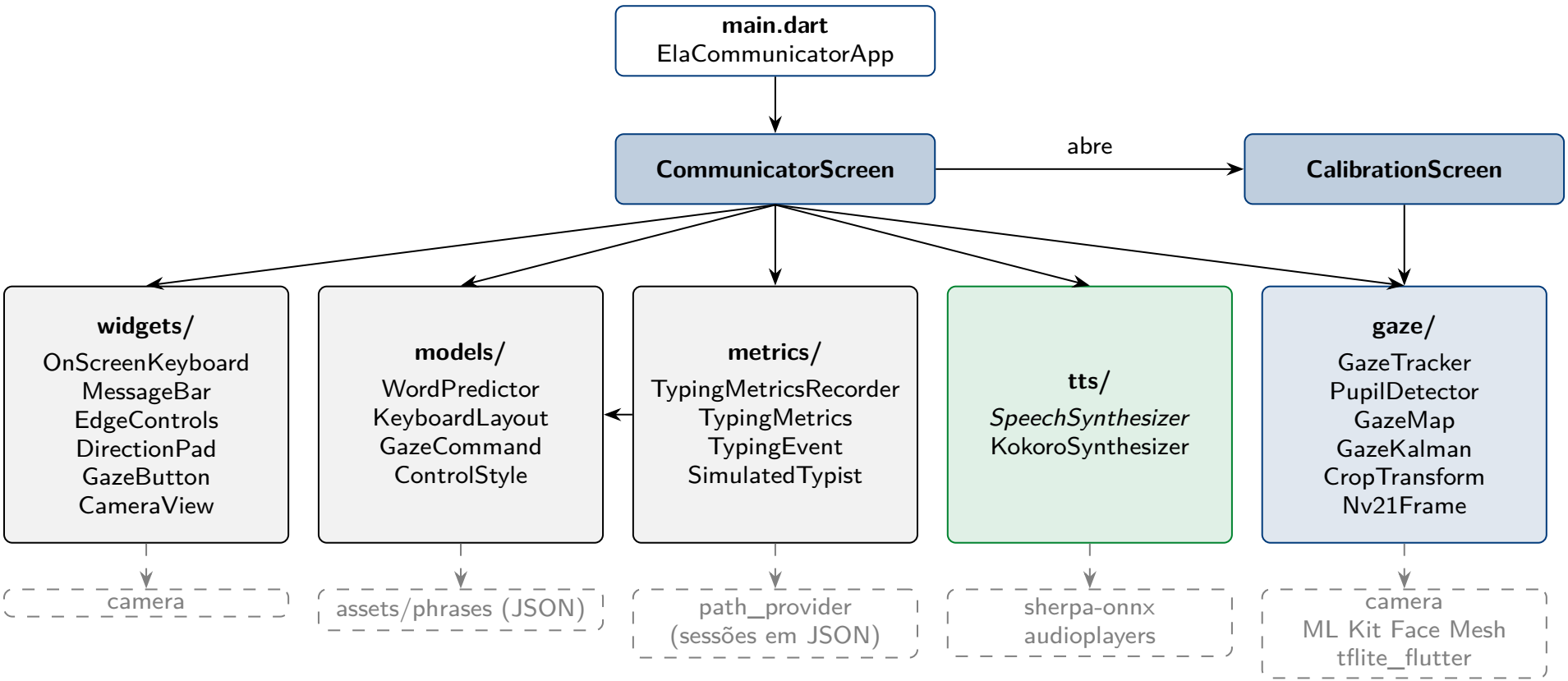
Android

- Tablets comuns custam bem menos que os aparelhos dedicados
- Câmera frontal e tela grande, sem hardware extra
- O Face Mesh do ML Kit está disponível no Android
- Instalação direta do APK, sem depender de loja

Setup e ferramentas

- IDE: VS Code, com as extensões de Dart e Flutter
- Depuração no tablet por USB; `flutter test` para geometria, calibração e rotação
- Treino do detector de pupila à parte, em Python com PyTorch e Ultralytics

Módulos


main.dart
ElaCommunicatorApp
CommunicatorScreen
abre
CalibrationScreen
widgets/
OnScreenKeyboard
MessageBar
EdgeControls
DirectionPad
GazeButton
CameraView
models/
WordPredictor
KeyboardLayout
GazeCommand
ControlStyle
metrics/
TypingMetricsRecorder
TypingMetrics
TypingEvent
SimulatedTypist
tts/
SpeechSynthesizer
KokoroSynthesizer
gaze/
GazeTracker
PupilDetector
GazeMap
GazeKalman
CropTransform
Nv21Frame
camera
assets/phrases (JSON)
path_provider
(sessões em JSON)
sherpa-onnx
audioplayers
camera
ML Kit Face Mesh
tflite_flutter

Módulos

`widgets/`: componentes da interface

`OnScreenKeyboard`	Teclado com uma tecla destacada por vez; as setas movem o destaque
`MessageBar`	Mensagem em composição e sugestões de palavras
`EdgeControls`	Setas nas bordas da tela: máxima distância entre os alvos
`DirectionPad`	Setas em cruz: arranjo compacto
`GazeButton`	Botão comum aos dois arranjos, para comparar só a geometria
`CameraView`	Prévia da câmera para o enquadramento; entrega a câmera ao rastreador

Módulos

`models/`: dados e regras da interface

`GazeCommand`	Cima, baixo, esquerda, direita e selecionar: contrato único de entrada para olhar, toque e botões
`KeyboardLayout`	Teclado QWERTY com uma linha de ações: espaço, apagar, limpar e falar
`WordPredictor`	Sugere até 3 palavras por n-gramas com Stupid Backoff, a partir de um corpus de frases de comunicação alternativa
`ControlStyle`	Escolhe entre setas nas bordas ou em cruz, para medir qual funciona melhor

Módulos

`tts/`: síntese de voz

`SpeechSynthesizer`	Interface com `initialize`, `speak`, `stop` e o estado da fala; a tela depende só dela
`KokoroSynthesizer`	Implementação com Kokoro e sherpa-onnx, numa isolate dedicada; uma fala por vez

Por que uma interface

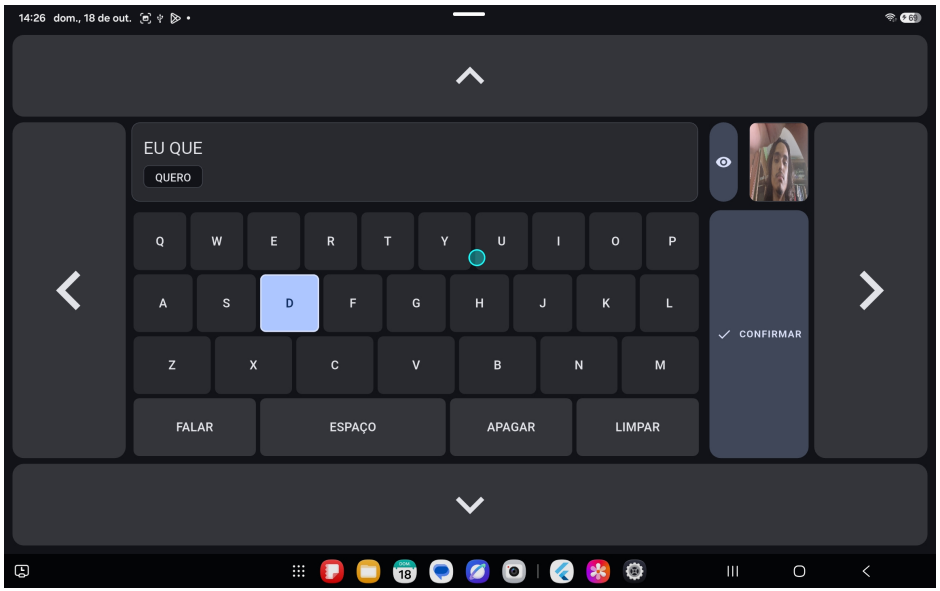
Nos testes, uma implementação falsa substitui o Kokoro, sem modelo nem plugins nativos.

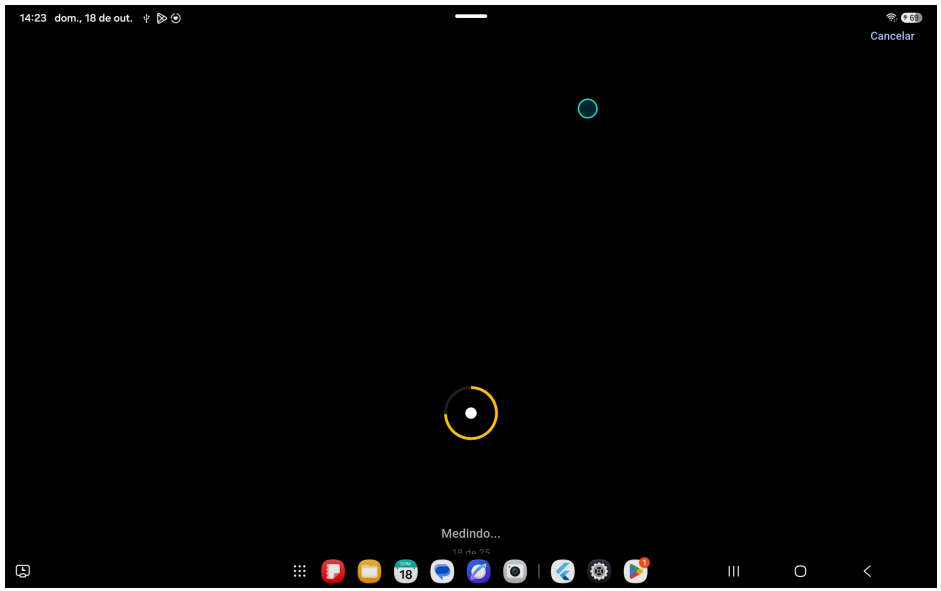
Módulos

gaze/: rastreamento do olhar

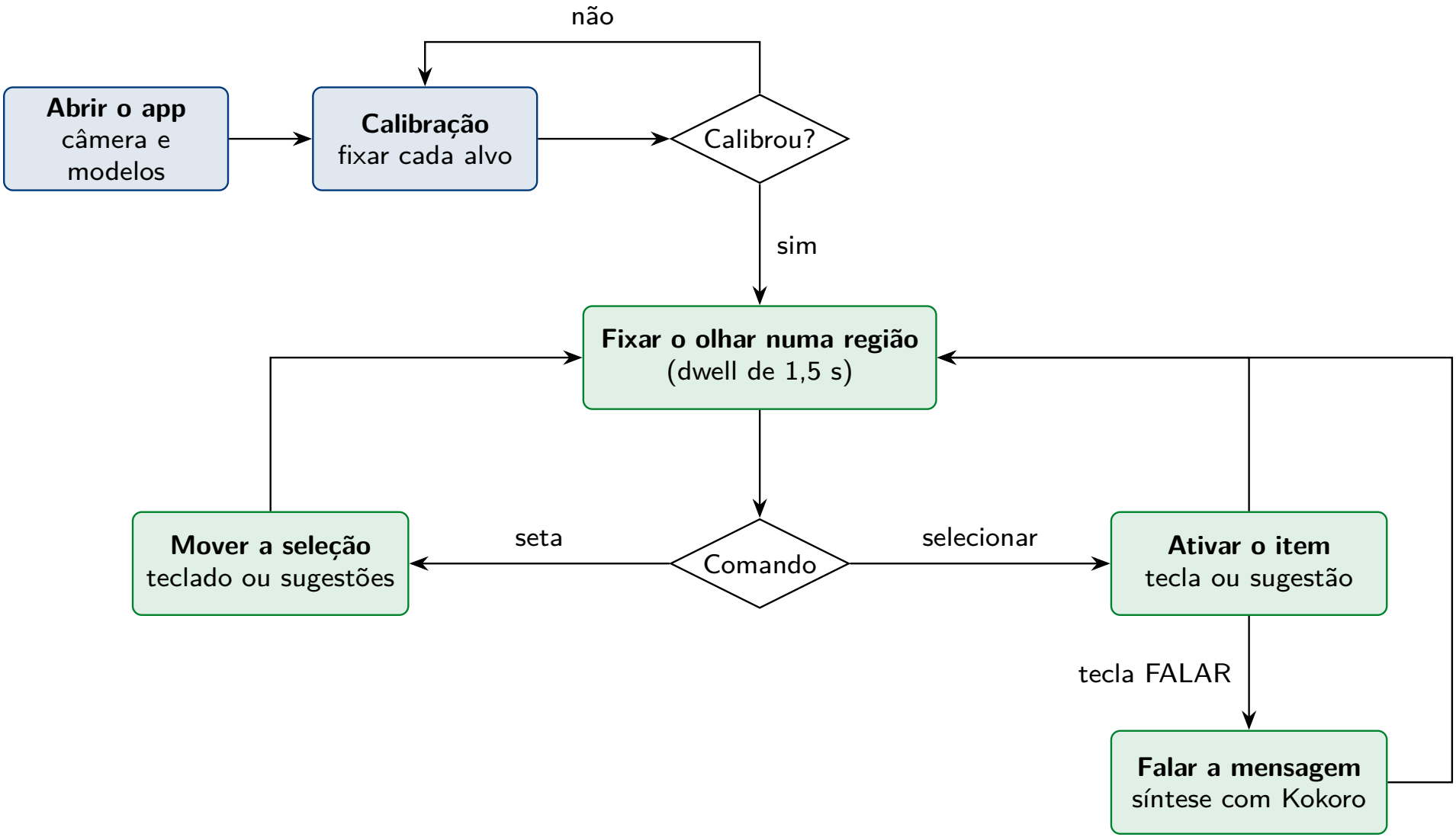
`GazeTracker`	Conduz o pipeline a cada quadro e emite o comando quando o dwell completa
`PupilDetector`	Roda o YOLO26n em TFLite e devolve o centro da pupila
`CropTransform`	Geometria do recorte do olho, idêntica à do treino
`Nv21Frame`	Recorta o olho direto do buffer da câmera, sem converter o quadro inteiro
`GazeMap`	Regressão do vetor pupila–canto para o ponto na tela
`GazeKalman`	Suaviza o ponto antes de virar comando

Telas


14:26 dom., 18 de out.
EU QUE
QUERO
Q W E R T Y U I O P
A S D F G H J K L
Z X C V B N M
FALAR
ESPAÇO
APAGAR
LIMPAR
CONFIRMAR


14:23 dom., 18 de out.
Cancelar
Medindo...

Fluxo de Uso do App



Métricas Obtidas com Pessoas

- Metodologia dos testes
- Participantes

Métricas Obtidas com Pessoas



Conclusão

- Síntese dos resultados

Trabalhos Futuros

- Melhoria 1
- Melhoria 2

Referências I

[1] Arizona Department of Education AT Short Term Loan Library. *Tobii Dynavox I-13 Communication Device*. URL: https://adeatloan.org/item/tobii-dynavox-i-13-communication-device-with-communicator-5-snap-scene-td-snap-td-control/ (acesso em 19/10/2026).

[2] Autor. *Referência sobre ELA*. 2026.

[3] Thorsten Brants et al. "Large Language Models in Machine Translation". Em: *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*. 2007, pp. 858–867. URL: https://aclanthology.org/D07-1090.

[4] Google. *Face mesh detection – ML Kit*. URL: https://developers.google.com/ml-kit/vision/face-mesh-detection (acesso em 19/10/2026).

[5] Google. *Flutter*. URL: https://flutter.dev (acesso em 18/10/2026).

[6] hexgrad. *Kokoro-82M*. URL: https://huggingface.co/hexgrad/Kokoro-82M (acesso em 26/10/2026).

Referências II

[7] k2-fsa. *sherpa-onnx*. URL: https://github.com/k2-fsa/sherpa-onnx (acesso em 26/10/2026).

[8] Rudolph Emil Kalman. "A New Approach to Linear Filtering and Prediction Problems". Em: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45. DOI: 10.1115/1.3662552.

[9] Vladimir I. Levenshtein. "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals". Em: *Soviet Physics Doklady* 10.8 (1966), pp. 707–710.

[10] Yinghao Aaron Li et al. "StyleTTS 2: Towards Human-Level Text-to-Speech through Style Diffusion and Adversarial Training with Large Speech Language Models". Em: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.

[11] Joseph Redmon et al. "You Only Look Once: Unified, Real-Time Object Detection". Em: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91.

[12] Smartbox Assistive Technology. *Lumin-i eye gaze technology*. URL: https://thinksmartbox.com/eye-gaze/lumin-i-camera/ (acesso em 19/10/2026).

Referências III

[13] Smartbox Assistive Technology. *Smartbox Pricing*. URL: https://smartboxaac.com/us/smartbox-pricing/ (acesso em 19/10/2026).

[14] R. William Soukoreff e I. Scott MacKenzie. "Metrics for Text Entry Research: An Evaluation of MSD and KSPC, and a New Unified Error Metric". Em: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*. 2003, pp. 113–120. DOI: 10.1145/642611.642632.

[15] Tobii Dynavox. *TD I-Series*. URL: https://www.tobiidynavox.com/products/td-i-series (acesso em 19/10/2026).

[16] Ultralytics. *Ultralytics YOLO26*. URL: https://docs.ultralytics.com/models/yolo26 (acesso em 19/10/2026).

[17] Xucong Zhang et al. "Appearance-Based Gaze Estimation in the Wild". Em: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 4511–4520.

Agradecimentos e Perguntas

Obrigado!

Perguntas?